

Manuale di Java

Docente: Carlo Leonardi

Cos'è Java

Java è un **linguaggio di programmazione** ad alto livello, orientato agli oggetti, progettato per essere **portabile, sicuro e robusto**

Java viene utilizzato in molti contesti, tra cui applicazioni desktop, applicazioni web, sistemi enterprise e sviluppo mobile.

Perche dovrei studiare java?

Studiare Java può essere estremamente vantaggioso per una serie di motivi, sia personali che professionali. Ecco alcuni dei motivi principali per cui dovresti considerare di imparare questo linguaggio di programmazione:

1. Versatilità e Diffusione:

- **Applicazioni aziendali:** Java è ampiamente utilizzato nello sviluppo di applicazioni aziendali, in particolare in ambienti bancari e finanziari.
- **Applicazioni Android:** Java è il linguaggio principale per lo sviluppo di applicazioni Android.
- **Backend development:** Molti sistemi backend, inclusi quelli su larga scala, utilizzano Java per la sua stabilità e scalabilità.

2. Robustezza e Affidabilità:

- **Gestione della memoria:** Java ha una gestione automatica della memoria (garbage collection) che riduce i rischi di memory leaks.
- **Sicurezza:** Java ha molte funzionalità di sicurezza integrate, rendendolo una scelta sicura per le applicazioni che richiedono protezione dei dati.

3. Elevata Domanda sul Mercato del Lavoro:

- **Opportunità di carriera:** La domanda di sviluppatori Java è alta, e molte grandi aziende cercano esperti in questo linguaggio.
- **Stabilità lavorativa:** Conoscere Java può offrire stabilità e una buona remunerazione, dato che molte organizzazioni lo utilizzano per applicazioni critiche.

4. Ecosistema Ricco:

- **Librerie e Framework:** Java ha una vasta gamma di librerie e framework (come Spring, Hibernate, e Apache Struts) che facilitano lo sviluppo di applicazioni complesse.
- **Tool di sviluppo:** Gli strumenti di sviluppo per Java, come IntelliJ IDEA, Eclipse e NetBeans, sono potenti e offrono molte funzionalità per migliorare la produttività del programmatore.

5. Community Attiva e Supporto:

- **Comunità globale:** La comunità di sviluppatori Java è vasta e attiva, il che significa che puoi

facilmente trovare aiuto, risorse e collaborazioni.

- **Documentazione e risorse di apprendimento:** Java è ben documentato e ci sono innumerevoli risorse di apprendimento disponibili, dai libri ai corsi online.

6. Portabilità e Compatibilità:

- **Piattaforma indipendente:** Grazie al motto "Write Once, Run Anywhere" (scrivi una volta, esegui ovunque), il codice Java può essere eseguito su qualsiasi dispositivo che supporta la Java Virtual Machine (JVM).

7. Base Solida per Altri Linguaggi:

- **Comprendere i concetti di programmazione orientata agli oggetti (OOP):** Java è un ottimo linguaggio per imparare i principi della programmazione orientata agli oggetti, che sono fondamentali anche in altri linguaggi come C#, Python, e C++.

Studiare Java ti fornisce quindi una solida base di competenze che sono altamente richieste e che possono aprire molte opportunità nel mondo della tecnologia e dello sviluppo software.

Tip

Java è un ottimo linguaggio per imparare a studiare la programmazione in generale.

Write Once, Run Anywhere

Scrivi una volta, esegui ovunque.

Verbosità:

La **verbosità** di un linguaggio di programmazione si riferisce alla quantità di codice necessario per esprimere un'idea, un'operazione o una funzionalità. In altre parole, è una misura di quanto dettagliatamente il programmatore deve specificare le istruzioni per ottenere un determinato risultato.

Un linguaggio è considerato **verboso** quando richiede molte righe di codice e dettagli espliciti per compiere operazioni relativamente semplici. Al contrario, un linguaggio meno verboso consente di esprimere le stesse operazioni con meno codice, spesso utilizzando sintassi più concisa e costrutti di alto livello.

Esempio di verbosità:

- **Java (Verboso):**

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

- **Python (Meno verboso):**

```
print("Hello, World!")
```

In questo esempio, per stampare "Hello, World!" a schermo, Java richiede più righe di codice rispetto a Python, che è molto più conciso.

La verbosità non è necessariamente un difetto; in alcuni casi, la verbosità può migliorare la leggibilità e la manutenibilità del codice, specialmente in progetti complessi dove la chiarezza del flusso di esecuzione è cruciale. Tuttavia, in altri contesti, una sintassi più concisa può rendere il codice più facile e veloce da scrivere e leggere.

Ecco una classifica generale dei linguaggi di programmazione basata sulla loro verbosità, con quelli più verbosi elencati per primi:

1. **Assembly Language:** È il linguaggio di programmazione più vicino all'architettura del computer e richiede spesso un gran numero di istruzioni per compiere anche semplici operazioni, rendendolo estremamente verboso.
2. **C/C++:** Questi linguaggi sono noti per essere più verbosi rispetto ad alcuni altri linguaggi ad alto livello. Possono richiedere una quantità significativa di codice per compiti comuni, soprattutto a causa della gestione manuale della memoria e delle dichiarazioni esplicite.
3. **Java:** Java è considerato più verboso rispetto a linguaggi come Python o JavaScript, principalmente a causa della sua sintassi più rigorosa e delle dichiarazioni di tipo esplicite. Tuttavia, fornisce una maggiore sicurezza e robustezza attraverso il controllo dei tipi.
4. **C#:** Simile a Java, C# richiede una certa quantità di codice per compiti comuni, ma offre un'ampia libreria standard e una forte integrazione con l'ecosistema di sviluppo di Microsoft.
5. **Objective-C:** Questo linguaggio è noto per la sua verbosità, specialmente quando confrontato con Swift, il suo successore. Objective-C richiede spesso la scrittura di una grande quantità di codice per compiti comuni, ma è ancora ampiamente utilizzato nell'ecosistema iOS/macOS.
6. **Swift:** Successore di Objective-C per lo sviluppo di applicazioni iOS/macOS, Swift è generalmente considerato meno verboso e più espressivo rispetto al suo predecessore.
7. **Ruby:** Ruby è noto per la sua filosofia di "convenzione sopra configurazione" che mira a ridurre la verbosità del codice. Tuttavia, può essere più verboso rispetto ad alcuni linguaggi più moderni come Python o JavaScript.
8. **Python:** Python è noto per la sua sintassi pulita e concisa, che spesso richiede meno codice per compiti comuni rispetto ad altri linguaggi. Questo lo rende generalmente meno verboso.
9. **JavaScript:** JavaScript, specialmente con l'uso di moderni framework come React o Angular, può essere molto espressivo e richiedere meno codice per ottenere gli stessi risultati di altri linguaggi.
10. **Scala/Kotlin:** Questi linguaggi, che girano sulla JVM, sono noti per offrire una sintassi più concisa rispetto a Java, riducendo la verbosità del codice.

Ricorda che questa classifica è soggettiva e dipende dall'esperienza e dalle preferenze personali dei programmatori. Alcuni potrebbero preferire linguaggi più verbosi per la loro chiarezza e leggibilità, mentre altri potrebbero preferire linguaggi meno verbosi per la loro efficienza e concisione.

Quanto è più verboso un linguaggio, maggiore sarà il tempo per poterlo scrivere, per poterlo debuggare.

Caratteristiche principali di Java

Java presenta diverse caratteristiche che lo rendono uno dei linguaggi più diffusi:

- **Orientato agli oggetti**: consente di modellare entità reali attraverso classi e oggetti.
- **Portabilità**: un programma compilato in Java può essere eseguito su sistemi diversi.
- **Robustezza**: offre controllo sui tipi e gestione automatica della memoria.
- **Sicurezza**: riduce molti errori comuni presenti in linguaggi meno controllati.
- **Semplicità**: la sintassi è pensata per essere leggibile e organizzata.

Queste proprietà rendono Java adatto anche a chi si avvicina per la prima volta alla programmazione.

Cos'è la JVM

La *Java Virtual Machine (JVM)* è il componente che permette di eseguire i programmi Java.

Quando si scrive un programma in Java, il codice non viene eseguito direttamente dal sistema operativo, ma viene prima compilato in un formato intermedio chiamato **bytecode**.

Sarà poi la JVM a leggere ed eseguire questo bytecode, rendendo possibile l'esecuzione dello stesso programma su computer diversi.

In questo modo Java garantisce una forte **indipendenza dalla piattaforma**.

Differenza tra file `.java` e file `.class`

In Java esistono due tipi di file fondamentali:

- **File `.java`**: contengono il **codice sorgente** scritto dal programmatore.
- **File `.class`**: contengono il **bytecode** generato dal compilatore Java.

Ad esempio:

- `Main.java` → file scritto dal programmatore
- `Main.class` → file generato dopo la compilazione

Il file `.java` è leggibile e modificabile da una persona, mentre il file `.class` è pensato per essere eseguito dalla JVM.

Come trasformare un file `.java` in un file `.class`

Per compilare un programma Java è necessario utilizzare il comando `javac`.

Esempio:

```
javac Main.java
```

```
Main.class
```

Successivamente, per eseguire il programma, si utilizza il comando:

```
java Main
```

È importante notare che nel comando di esecuzione **non va scritto** `.class`.

Il flusso corretto è quindi:

1. Scrivere il file `Main.java`
2. Compilare con `javac Main.java`
3. Eseguire con `java Main`

Cosa serve per eseguire un programma Java

Per scrivere, compilare ed eseguire un programma Java è necessario installare il **JDK** (*Java Development Kit*).

Il JDK include:

- il **compilatore** `javac`
- la **JVM**
- gli strumenti di sviluppo principali

Senza il JDK non è possibile trasformare un file `.java` in un file `.class`.

Per chi deve soltanto eseguire programmi Java, in alcuni contesti può bastare il JRE (*Java Runtime Environment*), ma normalmente nei corsi base si installa direttamente il JDK, perché permette sia di compilare sia di eseguire.

Dopo l'installazione del JDK, è possibile verificare il corretto funzionamento con i comandi:

```
java --version  
javac --version
```

Passaggi della Compilazione di Java in Bytecode

1. Scrittura del Codice Sorgente:

- Gli sviluppatori scrivono il codice Java utilizzando file con estensione `.java`.
- Esempio di un semplice programma Java:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

2. Compilazione del Codice Sorgente:

- Il codice sorgente Java viene compilato utilizzando il compilatore Java (`javac`).
- Esecuzione del comando di compilazione:

```
javac HelloWorld.java
```

- Il compilatore `javac` analizza il codice sorgente e lo traduce in bytecode, che è un formato intermedio indipendente dalla piattaforma.

3. Generazione del Bytecode:

- Il risultato della compilazione è un file `.class` contenente il bytecode.
- Per l'esempio sopra, il compilatore genera un file chiamato `HelloWorld.class`.
- Il bytecode è una rappresentazione binaria del programma che può essere eseguita dalla Java Virtual Machine (JVM).

Struttura del Bytecode

- Il bytecode è progettato per essere eseguito dalla JVM, che è disponibile per molte piattaforme hardware e sistemi operativi diversi.
- Il bytecode è indipendente dalla piattaforma, il che significa che lo stesso file `.class` può essere eseguito su qualsiasi sistema che abbia una JVM.

Esecuzione del Bytecode

1. Java Virtual Machine (JVM):

- La JVM è un interprete di bytecode che esegue i file `.class`.
- Quando si esegue un programma Java, la JVM carica il bytecode, lo verifica e lo esegue.
- Esecuzione del programma:

```
java HelloWorld
```

2. Interpretazione e Ottimizzazione:

- La JVM interpreta il bytecode riga per riga.
- Per migliorare le prestazioni, molte JVM moderne utilizzano la compilazione Just-In-Time (JIT), che traduce parti del bytecode in codice macchina nativo durante l'esecuzione.
- Questo codice nativo viene quindi eseguito direttamente dalla CPU, migliorando significativamente le prestazioni.

I compilatori ottimizzano il codice attraverso una serie di tecniche che mirano a migliorare l'efficienza del codice generato in termini di velocità di esecuzione, utilizzo della memoria e altre risorse. Queste ottimizzazioni possono essere effettuate durante diverse fasi del processo di compilazione. Ecco una panoramica delle principali tecniche di ottimizzazione utilizzate dai compilatori:

Fasi di Ottimizzazione

1. Ottimizzazione Locale:

- **Costant Folding:** Sostituisce le espressioni costanti con i loro valori calcolati a tempo di compilazione.

```
int x = 3 + 5; // diventa int x = 8;
```

- **Strength Reduction:** Sostituisce operazioni costose con operazioni equivalenti meno costose.

```
for (int i = 0; i < n; i++) {  
    a = i * 2; // può diventare a = i << 1; (shift a sinistra)  
}
```

2. Ottimizzazione Globale:

- **Common Subexpression Elimination (CSE):** Rimuove le ripetizioni di espressioni calcolate più volte.

```
int a = b * c + d;  
int e = b * c + f; // diventa int tmp = b * c; int a = tmp + d; int e = tmp + f;
```

- **Dead Code Elimination:** Rimuove il codice che non influisce sull'output del programma.

```
int x = 10;  
x = 20; // la prima assegnazione a x è inutile e viene rimossa.
```

3. Ottimizzazione Interprocedurale:

- **Inline Expansion:** Sostituisce le chiamate di funzione con il corpo della funzione stessa, riducendo l'overhead della chiamata di funzione.

```
inline int square(int x) {
    return x * x;
}
int y = square(5); // diventa int y = 5 * 5;
```

- **Function Cloning:** Crea versioni ottimizzate di funzioni per casi specifici di utilizzo.

4. Ottimizzazione a Livello di Ciclo:

- Loop Unrolling

: Riduce il numero di iterazioni di un ciclo, eseguendo più operazioni per iterazione.

```
for (int i = 0; i < n; i++) {
    a[i] = b[i] + c[i];
}
// può diventare
for (int i = 0; i < n; i += 2) {
    a[i] = b[i] + c[i];
    a[i + 1] = b[i + 1] + c[i + 1];
}
```

- Loop Invariant Code Motion

: Sposta il codice invariante fuori dal ciclo.

```
for (int i = 0; i < n; i++) {
    x = y + z; // se y e z non cambiano, può diventare:
    for (int i = 0; i < n; i++) {}
    x = y + z;
}
```

Ottimizzazioni Specifiche della JVM

Per linguaggi come Java, la Java Virtual Machine (JVM) utilizza tecniche aggiuntive durante l'esecuzione:

- **Just-In-Time (JIT) Compilation:** Compila parti del bytecode in codice macchina nativo durante l'esecuzione per migliorare le prestazioni.
- **Adaptive Optimization:** Monitora il comportamento del programma durante l'esecuzione e ottimizza dinamicamente le parti del codice che vengono eseguite frequentemente.

Errori di Compilazione

Gli errori di compilazione si verificano quando il codice non rispetta le regole sintattiche e semantiche del linguaggio di programmazione. Questi errori vengono rilevati dal compilatore prima che il programma venga eseguito. Esempi comuni includono errori di sintassi, errori di tipo e dichiarazioni mancanti.

Caratteristiche degli Errori di Compilazione:

- **Messaggi del Compilatore:** Durante la compilazione, il compilatore fornisce messaggi di errore dettagliati che indicano il tipo di errore e la posizione nel codice sorgente.
- **Impossibilità di Creare l'Eseguibile:** Se ci sono errori di compilazione, il compilatore non può generare il file eseguibile o bytecode.
- **Errori Sintattici:** Errori come parentesi mancanti, errori di battitura nei nomi delle variabili, o errori nella struttura del codice.

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!"  
    }  
}
```

In questo esempio, manca una parentesi chiusa `)`.

- **Errori Semantici:** Errori legati alla logica del linguaggio, come il tentativo di assegnare un valore di tipo errato a una variabile.

```
int number = "123"; // Errore: assegnazione di una stringa a una variabile int.
```

Errori di Runtime

Gli errori di runtime si verificano durante l'esecuzione del programma, dopo che il codice è stato correttamente compilato. Questi errori possono causare l'arresto anomalo del programma o comportamenti imprevisti.

Caratteristiche degli Errori di Runtime:

- **Messaggi di Eccezione:** Durante l'esecuzione, il programma può generare eccezioni o errori che vengono segnalati tramite messaggi specifici.
- **Crash o Comportamenti Anomali:** Il programma può arrestarsi in modo anomalo o comportarsi in modo imprevisto.
- **Esempi Comuni di Errori di Runtime:**

- `NullPointerException`: Tentativo di utilizzare un oggetto non inizializzato (null).

```
String str = null;  
System.out.println(str.length()); // Lancia NullPointerException.
```

- `ArrayIndexOutOfBoundsException`: Accesso a un elemento di un array con un indice fuori dai limiti.

```
int[] arr = new int[5];  
System.out.println(arr[10]); // Lancia ArrayIndexOutOfBoundsException.
```

- `ArithmeticException`: Errori matematici come la divisione per zero.

```
int result = 10 / 0; // Lancia ArithmeticException.
```

Distinguere tra errori di compilazione e di runtime è essenziale per lo sviluppo efficace del software. Gli errori di compilazione sono rilevati prima dell'esecuzione e sono spesso più facili da risolvere grazie ai messaggi del compilatore. Gli errori di runtime, invece, richiedono tecniche di gestione delle eccezioni e debugging per essere identificati e corretti durante l'esecuzione del programma. Utilizzando strumenti appropriati e pratiche di sviluppo, è possibile ridurre al minimo entrambi i tipi di errori.

Compilatore ed Inteprete

La differenza principale tra un linguaggio compilato e un linguaggio interpretato risiede nel modo in cui il codice sorgente viene tradotto ed eseguito. Ecco una spiegazione dettagliata di ciascuno:

Linguaggio Compilato

- **Compilazione:** Un linguaggio compilato viene trasformato da un compilatore in un file eseguibile (codice macchina) prima di essere eseguito. Questo processo di traduzione avviene una sola volta.
- **Esecuzione:** Il programma eseguibile prodotto dal compilatore viene eseguito direttamente dal sistema operativo.
- **Esempi:** C, C++, Rust, Go.
- Vantaggi:
 - **Performance:** I programmi compilati generalmente hanno una performance migliore rispetto ai programmi interpretati perché il codice macchina è eseguito direttamente dalla CPU.
 - **Ottimizzazione:** I compilatori possono ottimizzare il codice per migliorare le prestazioni e l'efficienza.
- Svantaggi:
 - **Portabilità:** Il codice compilato è spesso specifico per una piattaforma o un'architettura, rendendo più difficile eseguire lo stesso programma su sistemi diversi senza ricompilazione.
 - **Tempo di sviluppo:** Il ciclo di sviluppo può essere più lento perché ogni modifica al codice sorgente richiede una nuova compilazione.

Linguaggio Interpretato

- **Interpretazione:** Un linguaggio interpretato viene eseguito da un interprete che legge e traduce il codice sorgente riga per riga in tempo reale.
- **Esecuzione:** L'interprete traduce e esegue il codice sorgente direttamente, senza produrre un file eseguibile intermedio.
- **Esempi:** Python, Ruby, JavaScript, PHP.
- Vantaggi:

- **Portabilità:** Il codice sorgente è spesso portabile tra diverse piattaforme senza modifiche, purché un interprete appropriato sia disponibile su ciascuna piattaforma.
- **Rapidità nello sviluppo:** Il ciclo di sviluppo è più rapido perché non è necessario compilare il codice ogni volta che si effettua una modifica. Questo facilita il debugging e la sperimentazione.
- Svantaggi:
 - **Performance:** I programmi interpretati tendono a essere più lenti rispetto a quelli compilati perché l'interprete traduce il codice in tempo reale.
 - **Ottimizzazione limitata:** Gli interpreti non ottimizzano il codice nello stesso modo in cui fanno i compilatori.

Linguaggi con Compilazione Intermedia

Alcuni linguaggi utilizzano un approccio ibrido che combina compilazione e interpretazione:

- **Java:** Il codice Java viene compilato in bytecode, un formato intermedio, che viene poi eseguito dalla Java Virtual Machine (JVM). La **JVM** può interpretare il bytecode o utilizzare tecniche come Just-In-Time (JIT) compilation per migliorare le prestazioni.
- **C#:** Simile a Java, il codice C# viene compilato in un formato intermedio (Common Intermediate Language, CIL) ed eseguito dal runtime .NET.

La scelta tra un linguaggio compilato e uno interpretato dipende dai requisiti specifici del progetto, tra cui le esigenze di performance, portabilità, velocità di sviluppo e ambiente di esecuzione. Ciascuna tipologia ha i propri punti di forza e debolezza, che devono essere valutati in base al contesto del progetto.

Tip

Java si può considerare un linguaggio compilato, mentre bytecode può essere considerato un programma interpretato.

Struttura di un programma Java

Un programma Java è organizzato in **classi**.

Ogni applicazione Java contiene almeno una classe, e molto spesso include un metodo principale chiamato `main`, che rappresenta il punto di ingresso del programma.

Esempio:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Metodo main in Java

Il metodo `main` è il **punto di ingresso** di un programma Java.

È il primo metodo che viene eseguito quando si avvia un'applicazione.

```
public static void main(String[] args) {  
    System.out.println("Program started");  
}
```

Quanti metodi main possono esserci

In Java possono esistere **più metodi `main`**, ma con alcune regole importanti:

- ✓ Possono esistere **in classi diverse**
- ✗ Non possono esistere più `main` identici nella stessa classe

Esempio corretto:

```
class First {  
    public static void main(String[] args) {  
        System.out.println("First main");  
    }  
}  
  
class Second {  
    public static void main(String[] args) {  
        System.out.println("Second main");  
    }  
}
```

Quale main viene eseguito

Quando esegui un programma, viene avviato **solo il `main` della classe specificata**.

Esempio:

```
java First
```

➡ esegue il `main` della classe `First`

```
java Second
```

➡ esegue il `main` della classe `Second`

Argomenti della Linea di Comando

Il parametro `String[] args` consente di passare argomenti alla linea di comando al programma Java. Ad esempio:

```
public class CommandLineExample {  
    public static void main(String[] args) {  
        if (args.length > 0) {  
            System.out.println("Argomento passato: " + args[0]);  
        } else {  
            System.out.println("Nessun argomento passato.");  
        }  
    }  
}
```

Per passare più parametri al metodo `main` di Java da terminale, è necessario separare ogni parametro con spazi. Quando si esegue il programma Java dalla riga di comando, gli argomenti passati vengono inseriti nell'array `args` del metodo `main`. Ecco un esempio di come farlo:

Supponiamo di avere il seguente programma Java:

```
public class Main {  
    public static void main(String[] args) {  
        // Stampa tutti gli argomenti passati  
        for (String arg : args) {  
            System.out.println(arg);  
        }  
    }  
}
```

Per passare più parametri al metodo `main`, puoi eseguire il programma dalla riga di comando e inserire gli argomenti separati da spazi. Ad esempio:

```
java Main arg1 arg2 arg3
```

In questo esempio, `arg1`, `arg2` e `arg3` sono gli argomenti passati al metodo `main`. Essi verranno poi stampati dal programma.

Puoi passare qualsiasi numero di argomenti in questo modo. Gli argomenti passati saranno quindi accessibili tramite l'array `args` del metodo `main`, con ogni elemento dell'array che corrisponde a uno degli argomenti passati.

Il passaggio dei parametri serve ad evitare problemi di hardcoding.

Il termine **hardcoded** (o **codice hardcoded**) si riferisce a valori, comportamenti o logiche che sono direttamente scritti nel codice sorgente di un programma anziché essere determinati o configurati esternamente o dinamicamente. Questi valori sono fissi e non possono essere modificati senza cambiare il codice stesso e ricompilare o rieseguire il programma.

Caratteristiche del Codice Hardcoded:

- **Fissità:** I valori o le logiche sono immutabili a meno che non si modifichi il codice sorgente.
- **Difficoltà di Manutenzione:** Modificare questi valori richiede la modifica e la ricompilazione del codice, il che può essere poco pratico e incline a errori.
- **Mancanza di Flessibilità:** I programmi con codice hardcoded sono meno adattabili a nuovi requisiti o contesti operativi.

Esempi di Codice Hardcoded:

1. Valori Costanti:

```
public class Config {  
    public static void main(String[] args) {  
        int maxConnections = 10; // Questo valore è hardcoded.  
        System.out.println("Max Connections: " + maxConnections);  
    }  
}
```

2. Stringhe Fisse:

```
public class Greeting {  
    public static void main(String[] args) {  
        String greeting = "Hello, World!"; // Questa stringa è hardcoded.  
        System.out.println(greeting);  
    }  
}
```

3. Indirizzi di Server o File di Configurazione:

```
public class DatabaseConfig {  
    public static void main(String[] args) {  
        String dbUrl = "jdbc:mysql://localhost:3306/mydatabase"; // URL del  
        database hardcoded.  
        String username = "admin"; // Nome utente hardcoded.  
        String password = "password123"; // Password hardcoded.  
    }  
}
```

Problemi del Codice Hardcoded:

- **Manutenzione:** Ogni volta che un valore deve essere cambiato, il codice sorgente deve essere modificato e il programma ricompilato. Questo può portare a errori, specialmente in applicazioni grandi o complesse.
- **Flessibilità:** Applicazioni con molto codice hardcoded sono meno adattabili a nuovi contesti o requisiti. Ad esempio, cambiare l'URL di un database richiede modifiche al codice.
- **Sicurezza:** Hardcoding di informazioni sensibili come password o chiavi di accesso può esporre l'applicazione a rischi di sicurezza.

Soluzione

Per passare più parametri al metodo `main`, puoi eseguire il programma dalla riga di comando e inserire gli argomenti separati da spazi. Ad esempio:

```
java Main jdbc:mysql://localhost:3306/mydatabase admin password123
```

```
public class DatabaseConfig {  
    public static void main(String[] args) {  
        String dbUrl = args[0];  
        String username = args[1];  
        String password = args[2];  
    }  
}
```

Attributi di una classe

Gli **attributi** (detti anche *campi* o *variabili di istanza*) rappresentano le **caratteristiche** di un oggetto.

Servono a descrivere lo **stato** di un oggetto.

Esempio: una classe `Car` può avere attributi come marca, colore e velocità.

Definizione degli attributi

Gli attributi vengono dichiarati all'interno della classe, ma **fuori dai metodi**.

```
class Car {  
    String brand;  
    String color;  
    int speed;  
}
```

In questo esempio:

- `brand`, `color` e `speed` sono attributi
- ogni oggetto `Car` avrà i propri valori per questi attributi

Creazione e utilizzo degli attributi

Per utilizzare gli attributi è necessario creare un oggetto.

```
Car c = new Car();

c.brand = "Toyota";
c.color = "Red";
c.speed = 100;
```

Qui ogni oggetto mantiene i propri valori indipendentemente dagli altri.

Accesso agli attributi

Gli attributi si accedono tramite la seguente sintassi:

```
nomeOggetto.nomeAttributo
```

Esempio:

```
System.out.println(c.brand);
```

Visibilità degli attributi

Gli attributi possono avere diversi livelli di visibilità:

- `public` → accessibili da qualsiasi classe
- `private` → accessibili solo all'interno della classe
- `protected` → accessibili nella stessa classe e nelle sottoclassi

Esempio:

```
class Car {
    public String brand;
    private int speed;
}
```

Attributi di istanza

Gli attributi definiti senza `static` sono chiamati **attributi di istanza**.

Questo significa che:

- ogni oggetto ha una propria copia dell'attributo
- i valori possono essere diversi tra oggetti

Esempio:

```
Car c1 = new Car();
Car c2 = new Car();

c1.brand = "Toyota";
c2.brand = "BMW";
```

Costruttori (Constructors)

Un **costruttore** è un blocco di codice speciale usato per **creare e inizializzare un oggetto**.

Signature del costruttore

La **firma (signature)** di un costruttore è composta da:

- Nome della classe
- Lista dei parametri (numero, tipo e ordine)

👉 **NON include:**

- Tipo di ritorno (infatti non esiste)
- Nome dei parametri (solo i tipi contano)

Esempio:

```
class Person {
    String name;
    int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Signature:

```
Person(String, int)
```

👉 Nota:

- Se cambi l'ordine o il tipo dei parametri → cambia la signature
- Questo permette il **costruttore overload**

Overloading:

```
class Person {  
    public String name;  
  
    public Person() {  
        this.name = "Unknown";  
    }  
  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

Signature:

- `Person()`
- `Person(String)`

Metodi (Methods)

Un **metodo** è una funzione che definisce il **comportamento** di un oggetto.

Signature del metodo

La **firma (signature)** di un metodo include:

- **Nome del metodo**
- **Lista dei parametri (tipo, numero, ordine)**

👉 **NON include:**

- Tipo di ritorno
- Nome dei parametri

Esempio:

```
class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Signature:

```
add(int, int)
```

Attenzione importante

Il **tipo di ritorno NON** fa parte della signature.

Quindi questo codice è **errore**:

```
class Test {  
    public int sum(int a, int b) {  
        return a + b;  
    }  
  
    public double sum(int a, int b) { // ✗ errore  
        return a + b;  
    }  
}
```

👉 Perché la signature è la stessa:

```
sum(int, int)
```

Overloading dei metodi

Puoi avere più metodi con lo stesso nome **solo se cambia la signature**.

Esempio valido:

```
class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public double add(double a, double b) {  
        return a + b;  
    }  
}
```

Signature:

- `add(int, int)`
- `add(double, double)`

Riassunto finale

Costruttori:

- Nome = nome classe
- Nessun tipo di ritorno
- Signature = nome classe + parametri

Metodi:

- Nome libero
- Hanno tipo di ritorno
- Signature = nome metodo + parametri

👉 In entrambi i casi:

✓ conta **tipo, numero e ordine dei parametri**

✗ non conta il tipo di ritorno

Visibilità degli attributi

In Java, gli attributi possono avere diversi livelli di visibilità.

I principali sono:

- `public`
 - `private`
 - `protected`
 - *(default)* (nessun modificatore)
-

Public

Un attributo `public` è accessibile da **qualsiasi classe**.

```
class Person {  
    public String name;  
}
```

👉 Può essere letto e modificato ovunque.

Private

Un attributo `private` è accessibile **solo all'interno della classe**.

```
class Person {  
    private int age;  
}
```

👉 Non può essere modificato direttamente dall'esterno.

Protected

Un attributo `protected` è accessibile:

- all'interno della **stessa classe**
- da tutte le classi nello **stesso package**

```
class Person {  
    protected String nationality;  
}
```

👉 Si comporta in modo simile al livello *default* (package).

Default (package)

Se non si specifica alcun modificatore, l'attributo è accessibile solo nel **package**.

```
class Person {  
    String surname;  
}
```

👉 Non è accessibile da classi in altri package.

Confronto

- `public` → ovunque
 - `private` → solo nella classe
 - `protected` → stessa classe + stesso package
 - `default` → stesso package
-

Buona pratica

In Java, gli attributi dovrebbero essere:

👉 quasi sempre `private`

Perché:

- si evita l'accesso diretto
 - si proteggono i dati
 - si mantiene il controllo sui valori
-

Getter e Setter

Per accedere agli attributi `private` si utilizzano i **getter** e i **setter**.

- **Getter** → permettono di leggere il valore
 - **Setter** → permettono di modificarlo
-

Esempio

```
class Person {  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {
```

```
        this.age = age;
    }
}
```

Utilizzo

```
Person p = new Person();

p.setName("Luca");
p.setAge(25);

System.out.println(p.getName());
System.out.println(p.getAge());
```

Validazione

La **validazione** serve a controllare che il dato sia corretto prima di salvarlo.

Esempio:

```
public void setAge(int age) {
    if (age >= 0) {
        this.age = age;
    }
}
```

👉 Evita valori non validi (es. età negativa)

Sanificazione

La **sanificazione** serve a modificare il dato per renderlo corretto o uniforme.

Esempio:

```
public void setName(String name) {
    this.name = name.trim();
}
```

👉 Rimuove spazi inutili

Validazione + Sanificazione

Spesso si usano insieme:

```
public void setName(String name) {  
    if (name != null && !name.isEmpty()) {  
        this.name = name.trim();  
    }  
}
```

👉 Prima controllo (validazione), poi pulizia (sanificazione)

Perché usare getter & setter

I setter permettono di:

- controllare i dati
- evitare errori
- mantenere il codice pulito
- aggiungere logica (es. validazioni)
- centralizzare la logica sugli attributi

Keyword static in Java

Cos'è static

La keyword `static` in Java indica che un elemento appartiene alla **classe** e non alla singola istanza (oggetto).

👉 Significa che esiste **una sola copia condivisa** tra tutti gli oggetti.

Attributi statici

Un attributo `static` è condiviso da tutte le istanze della classe.

```
class Player {  
    private static int MAX_NUMBER_OF_SHIRT = 99;  
    private static int MIN_NUMBER_OF_SHIRT = 1;  
}
```

In questo caso:

- `MAX_NUMBER_OF_SHIRT` appartiene alla classe `Player`
- `MIN_NUMBER_OF_SHIRT` appartiene alla classe `Player`
- questi valori sono condivisi da tutti gli oggetti `Player`

Perché usare static in questo caso

Questi attributi rappresentano un'informazione **comune a tutti i giocatori**.

Infatti:

- il numero massimo di maglia non cambia da un oggetto all'altro
- il numero minimo di maglia non cambia da un oggetto all'altro
- non ha senso che ogni oggetto abbia una propria copia di questi valori

👉 Per questo motivo è corretto dichiararli `static`.

Accesso a un attributo statico

Gli attributi statici si accedono tramite il nome della classe:

```
System.out.println(Player.MAX_NUMBER_OF_SHIRT);  
System.out.println(Player.MIN_NUMBER_OF_SHIRT);
```

👉 Non è necessario creare un oggetto.

Static e visibilità

Nel nostro esempio gli attributi sono anche `private`:

```
private static int MAX_NUMBER_OF_SHIRT = 99;  
private static int MIN_NUMBER_OF_SHIRT = 1;
```

Questo significa che:

- appartengono alla classe
- non sono accessibili direttamente dall'esterno

👉 Per leggerli dall'esterno serve un metodo dedicato.

Esempio con getter statici

```
class Player {  
    private static int MAX_NUMBER_OF_SHIRT = 99;  
    private static int MIN_NUMBER_OF_SHIRT = 1;  
  
    public static int getMaxNumberOfShirt() {  
        return MAX_NUMBER_OF_SHIRT;  
    }  
  
    public static int getMinNumberOfShirt() {  
        return MIN_NUMBER_OF_SHIRT;  
    }  
}
```

Utilizzo

```
System.out.println(Player.getMaxNumberOfShirt());  
System.out.println(Player.getMinNumberOfShirt());
```

Differenza tra attributi statici e non statici

```
class Player {  
    private static int MAX_NUMBER_OF_SHIRT = 99;  
    private static int MIN_NUMBER_OF_SHIRT = 1;  
  
    private String name;  
    private String surname;  
    private int numberOfShirt;  
}
```

In questo esempio:

- `MAX_NUMBER_OF_SHIRT` e `MIN_NUMBER_OF_SHIRT` sono comuni a tutti i giocatori
- `name`, `surname` e `numberOfShirt` appartengono al singolo oggetto

👉 I dati comuni sono `static`, i dati specifici del giocatore no.

Quando usare static

Si usa `static` quando un dato:

- è comune a tutti gli oggetti
 - non dipende dalla singola istanza
 - deve esistere una sola volta nella classe
-

Riassunto

- `static` significa che un elemento appartiene alla **classe**
- esiste una sola copia condivisa
- nel caso di `Player`, i limiti del numero di maglia sono dati comuni
- per questo `MAX_NUMBER_OF_SHIRT` e `MIN_NUMBER_OF_SHIRT` sono correttamente dichiarati `static`

Tipi primitivi e oggetti in Java

Tipi di dato in Java

In Java esistono due grandi categorie di tipi:

- **Tipi primitivi**
 - **Oggetti (tipi reference)**
-

Tipi primitivi

I tipi primitivi rappresentano valori semplici.

I principali sono:

- `int` → numeri interi
- `double` → numeri decimali
- `char` → caratteri
- `boolean` → vero/falso

```
int age = 25;
double price = 19.99;
char initial = 'A';
boolean isStudent = true;
```

👉 Contengono direttamente il valore.

Oggetti (Reference Type)

Gli oggetti sono istanze di una classe.

Esempio:

```
class Player {  
    String name;  
}
```

Creazione:

```
Player p = new Player();  
p.name = "Luca";
```

👉 In questo caso la variabile contiene un **riferimento all'oggetto**, non il valore diretto.

Differenza principale

- **Primitivi** → contengono il valore
- **Oggetti** → contengono un riferimento (indirizzo in memoria)

Esempio:

```
int a = 10;  
int b = a;  
  
b = 20;  
System.out.println(a); // 10  
Player p1 = new Player();  
Player p2 = p1;  
  
p2.name = "Marco";  
System.out.println(p1.name); // Marco
```

👉 Nei primitivi si copia il valore, negli oggetti si copia il riferimento.

La classe String

`String` è una **classe**, quindi è un oggetto.

Esempio:

```
String name = "Luca";
```

👉 Anche se sembra un primitivo, in realtà è un oggetto.

Perché String sembra un primitivo

La classe `String` ha alcune caratteristiche che la fanno sembrare un tipo primitivo:

- si può dichiarare senza `new`
- ha una sintassi semplice
- è molto utilizzata

Esempio:

```
String s1 = "Hello";  
String s2 = "Hello";
```

String è immutabile

Le stringhe in Java sono **immutabili**.

👉 Una volta create, non possono essere modificate.

Esempio:

```
String s = "Hello";  
s = s + " World";
```

👉 Non modifica la stringa originale, ma ne crea una nuova.

String vs primitivi

- come gli oggetti:
 - è una classe
 - ha metodi
 - usa riferimenti
- come i primitivi:
 - sintassi semplice
 - utilizzo immediato
 - comportamento intuitivo

Riassunto

- Java distingue tra **primitivi** e **oggetti**
- i primitivi contengono il valore
- gli oggetti contengono un riferimento
- `String` è un oggetto ma si usa in modo simile a un primitivo

- le stringhe sono **immutabili**

Perché String non richiede import

La classe `String` è una classe **built-in** di Java.

👉 Questo significa che è inclusa automaticamente nel linguaggio.

Package java.lang

`String` fa parte del package:

```
java.lang
```

Questo package viene:

👉 **importato automaticamente** in ogni programma Java

Esempio

```
String name = "Luca";
```

👉 Funziona senza import perché `String` è già disponibile.

Altri esempi di classi in java.lang

- `String`
- `Integer`
- `Double`
- `Math`
- `System`

👉 Tutte queste classi non richiedono import.

LocalDate invece?

La classe `LocalDate` appartiene al package:

```
java.time
```

👉 Questo package **NON** viene importato automaticamente

Utilizzo di LocalDate

Per usare `LocalDate` è necessario importarlo:

```
import java.time.LocalDate;

LocalDate today = LocalDate.now();
```

Perché questa differenza

Java importa automaticamente solo `java.lang` perché contiene:

- classi fondamentali
- tipi base del linguaggio
- strumenti utilizzati in quasi tutti i programmi

Gli altri package, come `java.time`, sono:

- più specifici
- non sempre necessari

👉 Per questo devono essere importati esplicitamente.

Concetto chiave

- `String` è in `java.lang` → import automatico
- `LocalDate` è in `java.time` → import manuale

👉 Non è una differenza tra le classi, ma tra i **package** a cui appartengono.

💡 Tip

DRY

Ripetere lo stesso codice in più punti porta a diversi problemi:

- codice più difficile da mantenere
- maggiore rischio di errori
- modifiche più complesse
- perdita di tempo

Variabili Primitive

1. Tipo di Dato Predefinito:

- Le variabili primitive rappresentano tipi di dati predefiniti in Java, come interi, booleani, caratteri, virgola mobile, ecc.

2. Memoria Diretta:

- Le variabili primitive memorizzano direttamente i valori effettivi dei dati nella memoria.

3. Efficienza di Memoria ed Esecuzione:

- Le variabili primitive sono più efficienti in termini di utilizzo della memoria e velocità di esecuzione, poiché occupano meno spazio e non richiedono l'allocazione dinamica della memoria.

4. Accesso Diretto:

- L'accesso alle variabili primitive avviene direttamente, senza passare attraverso un riferimento o un puntatore.

5. Tipo Valore:

- Le variabili primitive rappresentano un tipo valore, il che significa che quando vengono assegnate ad un'altra variabile, il valore viene copiato.

Esempio:

```
int x = 2;
int y = x; // copia per valore
x = 3;
System.out.println(y); // y vale 2
```

Variabili Non Primitive (Oggetti)

1. Tipo di Dato Definito dall'Utente:

- Le variabili non primitive (oggetti) rappresentano tipi di dati definiti dall'utente o forniti dalle librerie Java, come classi, interfacce o array.

2. Riferimento all'Area di Memoria:

- Le variabili non primitive memorizzano un riferimento (o puntatore) all'area di memoria dove sono memorizzati effettivamente i dati.

3. Allocazione Dinamica della Memoria:

- Gli oggetti richiedono l'allocazione dinamica della memoria e possono essere istanziati in modo dinamico durante l'esecuzione del programma.

4. Accesso Indiretto:

- L'accesso ai dati delle variabili non primitive avviene indirettamente, attraverso il riferimento all'oggetto.

5. Tipo Riferimento:

- Le variabili non primitive rappresentano un tipo riferimento, il che significa che quando vengono assegnate ad un'altra variabile, viene copiato solo il riferimento all'oggetto, non l'oggetto stesso. Entrambe le variabili puntano allo stesso oggetto in memoria.

Esempio:

```
String[] arr = { "pippo", "caio" };
String[] arr2 = arr; // copia per riferimento
arr[0] = "sempronio";
System.out.println(arr2[0]); // verrà stampato sempronio
```

Casting (o Parsing)

Il casting in Java si riferisce alla conversione di un tipo di dato in un altro tipo di dato compatibile. Può essere necessario quando si lavora con variabili di tipo diverso o quando si utilizzano metodi che restituiscono un tipo di dato diverso da quello desiderato. Il casting può essere effettuato in due modi: esplicito (casting) e implicito (promotion).

Casting Esplicito

Il casting esplicito avviene quando si forza la conversione di un tipo di dato in un altro tipo di dato compatibile. Viene eseguito mettendo tra parentesi il tipo di dato a cui si desidera convertire. Ecco un esempio:

```
double d = 3.14;
int i = (int) d; // Conversione esplicita di double a int
```

In questo esempio, `d` è di tipo `double` e viene convertito in un `int`. Poiché un `int` non può memorizzare valori decimali, il risultato sarà troncato e assegnato a `i`.

Casting Implicito (Promotion)

Il casting implicito, noto anche come promotion, avviene quando si converte automaticamente un tipo di dato di dimensione più piccola in un tipo di dato di dimensione più grande senza perdita di precisione. Questo avviene, ad esempio, quando si assegna un valore `int` a un `long`. Ecco un esempio:

```
int x = 10;
long y = x; // Conversione implicita di int a long
```

In questo caso, `x` è di tipo `int`, ma viene assegnato a una variabile di tipo `long`. Poiché `long` ha una dimensione maggiore di `int`, non si perderanno dati durante la conversione e avverrà in modo implicito.

Il casting da `String` a un tipo numerico è un'operazione comune in Java quando si desidera convertire una stringa che rappresenta un numero in un tipo di dato numerico, come `int`, `double`, `float`, ecc. Ci sono diversi modi per fare ciò, utilizzando metodi forniti dalle classi wrapper per i tipi primitivi o utilizzando metodi delle classi di utilità come `Integer`, `Double`, `Float`, ecc.

Ecco alcuni esempi di come eseguire il casting da `String` a tipi numerici:

Conversione con i Metodi delle Classi Wrapper:

```
String str = "123";
int intValue = Integer.parseInt(str);
double doubleValue = Double.parseDouble(str);
float floatValue = Float.parseFloat(str);
long longValue = Long.parseLong(str);
```

La principale differenza tra `double` e `float` in Java riguarda la precisione e la dimensione in bit di ciascun tipo di dato.

1. Precisione:

- `double` è un tipo di dato a virgola mobile a doppia precisione e offre una precisione maggiore rispetto a `float`.
- `float` è un tipo di dato a virgola mobile a precisione singola.

2. Dimensione in Bit:

- `double` è un tipo di dato a 64 bit.
- `float` è un tipo di dato a 32 bit.

3. Gamma di Valori:

- A causa della maggiore precisione, `double` può rappresentare un intervallo più ampio di valori rispetto a `float`.
- `float`, avendo meno bit a disposizione, ha una gamma di valori rappresentabili più limitata rispetto a `double`.

4. Utilizzo in Java:

- `double` è comunemente utilizzato per applicazioni che richiedono una maggiore precisione, come calcoli scientifici o finanziari.
- `float` è utilizzato in situazioni in cui la precisione maggiore di `double` non è necessaria e l'occupazione di memoria è una considerazione importante, ad esempio in applicazioni embedded o in dispositivi con risorse limitate.

5. Suffix:

- In Java, il valore letterale per `double` termina con la lettera `d` o `D`, mentre il valore letterale per `float` termina con la lettera `f` o `F`. Ad esempio, `3.14d` rappresenta un valore `double`, mentre `3.14f` rappresenta un valore `float`.

```
double doubleValue = 3.14d;
float floatValue = 3.14f;
```

In generale, è consigliabile utilizzare `double` se non ci sono restrizioni sulle risorse e se è necessaria una maggiore precisione. Tuttavia, se è necessario conservare la memoria o se la precisione è meno critica, `float` può essere una scelta più appropriata.

In Java, il tipo di dato `int` è un intero con segno a 32 bit. Questo significa che può rappresentare valori interi compresi tra -2^{31} (-2147483648) e $2^{31}-1$ (2147483647), inclusi questi due estremi.

Ecco un esempio di come viene dichiarato un `int` in Java:

```
int numero = 2147483647; // Massimo valore consentito per un int
```

Se si cerca di assegnare un valore superiore o inferiore ai limiti del tipo di dato `int`, verrà sollevato un errore di compilazione.

```
int numero = 2147483648; // Errore: Integer number too large
```

Per il tipo di dato `long` in Java, si hanno valori interi compresi tra -2^{63} (-9223372036854775808) e $2^{63}-1$ (9223372036854775807), inclusi questi due estremi. Il tipo `long` è un intero con segno a 64 bit.

Ecco un esempio di come viene dichiarato un `long` in Java:

```
long numeroLungo = 9223372036854775807L;
```

Il numero `double` più grande che è possibile rappresentare in Java dipende dalla specifica del linguaggio e dal sistema hardware su cui viene eseguito il programma. Tuttavia, il più grande numero rappresentabile come `double` in Java è approssimativamente $1.7976931348623157 \times 10^{308}$.

In Java, il tipo di dato `double` è un numero a virgola mobile a doppia precisione a 64 bit, che può rappresentare un'ampia gamma di valori, inclusi numeri molto grandi o molto piccoli. Il valore massimo positivo rappresentabile viene definito dalla costante `Double.MAX_VALUE`, che è approssimativamente $1.7976931348623157 \times 10^{308}$.

Ecco un esempio di come si può accedere al valore massimo di un `double` in Java:

```
double maxDouble = Double.MAX_VALUE;  
System.out.println(maxDouble); // Stampa: 1.7976931348623157E308
```

Altri esempi di conversione:

```
String x = String.valueOf(3);
boolean a = Boolean.parseBoolean("true");
String v = String.valueOf(true);
String z = true + "";
```

Convenzioni e nomenclatura

In Java, seguire convenzioni di denominazione coerenti e leggibili è fondamentale per mantenere il codice chiaro e facilmente mantenibile.

Di seguito sono riportate le convenzioni di denominazione generalmente accettate per i nomi delle classi, dei metodi e degli array:

Nomi delle classi

I nomi delle classi in Java dovrebbero essere in `PascalCase` (anche noto come `UpperCamelCase`), dove ogni parola inizia con una lettera maiuscola e non ci sono spazi tra le parole.

I nomi delle classi dovrebbero essere sempre dei sostantivi al singolare.

Esempi:

```
public class Employee {
    // classe Employee
}

public class DataProcessor {
    // classe DataProcessor
}

public class UserAccount {
    // classe UserAccount
}
```

Tip

Tieni in considerazione del loro utilizzo, che la sintassi sia coerente:

```
Employee employee = new Employee()
```

Non usare invece:

```
Employees employee = new Employees()
```

Nomi dei metodi

I nomi dei metodi dovrebbero essere in `camelCase`, dove la prima parola inizia con una lettera minuscola e ogni parola successiva inizia con una lettera maiuscola. I nomi dei metodi dovrebbero essere verbi o frasi verbali che descrivono l'azione che il metodo compie, tieni in considerazione anche cosa restituisce.

Esempi:

```
public class Calculator {  
  
    public int addNumbers(int a, int b) {  
        return a + b;  
    }  
  
    public void printResult() {  
        System.out.println("Result");  
    }  
  
    public boolean isPrime(int number) {  
        // logica per verificare se un numero è primo  
        return true;  
    }  
}
```

Tip

I metodi che tornano un valore booleano convenzionalmente iniziano con `is` o `has`.

Signature di un metodo

```
modificatore_di_visibilità tipo_di_ritorno nome_del_metodo (tipo_input nome_input, ...)
```

Nomi degli array

I nomi degli array dovrebbero essere descrittivi e utilizzare lo stile `camelCase`, come per le altre variabili. Di solito, i nomi degli array sono sostantivi plurali che indicano che si tratta di una raccolta di elementi.

Esempi:

```
public class ArrayExample {  
  
    private int[] numbers = new int[3];  
    private String[] names;
```

```

private double[] values;

public void initializeArrays() {
    numbers = new int[4];
    names = new String[] { "Alice", "Bob", "Charlie" };
    values = new double[] { 10.5, 20.5, 30.5 };
}

public void printArray(int[] array) {
    for (int num : array) {
        System.out.println(num);
    }
}
}

```

Certo, ecco come dovresti denominare le proprietà (o variabili) in Java, seguendo le convenzioni comuni:

Nomi delle proprietà (o variabili)

I nomi delle proprietà (o variabili) dovrebbero essere in `camelCase`, come per i nomi dei metodi e degli array.

I nomi delle variabili dovrebbero essere descrittivi e specificare chiaramente lo scopo della variabile, usare dunque dei sostantivi.

Esempi:

```

public class Employee {
    private String firstName;
    private String lastName;
    private int employeeId;
    private double salary;
}

```

Riepilogo delle convenzioni di denominazione:

1. **Classi:** `PascalCase`

```
public class MyClassName { }
```

2. **Metodi:** `camelCase`

```
public void getSomething() { }
```

3. **Array:** `camelCase`, solitamente al plurale

```
private int[] values;
```

4. Proprietà (variabili di istanza): camelCase

```
private String firstName;
```

Seguendo queste convenzioni, il codice sarà più leggibile e in linea con le pratiche comuni della comunità Java.

Tip

Una variabile non può iniziare con un numero

Non essere il nome di una keyword.

Gestione degli Array in Java

Cos'è un array

Un array è una struttura dati che permette di memorizzare più valori dello stesso tipo.

Caratteristiche

- dimensione fissa
- accesso tramite indice (parte da 0)

Dichiarazione

```
int[] numbers = new int[5];  
String[] names = new String[3];
```

Valori di default

- int -> 0
- double -> 0.0
- boolean -> false
- oggetti -> null

Inizializzazione

```
int[] numbers = {10, 20, 30};  
String[] names = {"Mario", "Luigi", "Anna"};
```

Accesso

```
numbers[0] // primo elemento
```

Scrittura

```
numbers[0] = 100;
```

Array di primitivi

```
int[] ages = {18, 21, 25};
```

Array di oggetti

```
String[] names = {"Marco", "Luca"};
```

Array di oggetti custom

```
class Player {  
    String name;  
    int number;  
}  
  
Player[] players = new Player[2];
```

Cicli

for classico

```
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}
```

for-each

```
for (int n : numbers) {  
    System.out.println(n);  
}
```

Attenzione

- `ArrayIndexOutOfBoundsException`
- `NullPointerException`

Ereditarietà

Ereditarietà è un concetto fondamentale della programmazione orientata agli oggetti che permette di creare una nuova classe basata su una classe esistente. La nuova classe, chiamata sottoclasse o classe derivata, eredita attributi e metodi dalla classe esistente, chiamata superclasse o classe base. L'ereditarietà consente di riutilizzare il codice e di creare gerarchie di classi.

- **Superclasse (classe base):** La classe esistente dalla quale si ereditano attributi e metodi.
- **Sottoclasse (classe derivata):** La nuova classe che eredita attributi e metodi dalla superclasse.

In Java, l'ereditarietà si implementa usando la parola chiave `extends`.

```
class Animal {
    void eat() {
        System.out.println("The animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog is barking");
    }
}

public class TestInheritance {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Inherited from Animal class
        dog.bark(); // Specific to Dog class
    }
}
```

Polimorfismo

Polimorfismo è un altro concetto fondamentale della programmazione orientata agli oggetti che consente agli oggetti di essere trattati come istanze della loro superclasse piuttosto che della loro classe specifica. Questo permette di scrivere codice più generico e flessibile. In Java, il polimorfismo si ottiene attraverso metodi sovrascritti (overriding) e l'interfacciamento.

- **Sovrascrittura (overriding):** Consente a una sottoclasse di fornire una specifica implementazione di un metodo che è già definito nella sua superclasse.

```

class Animal {
    void makeSound() {
        System.out.println("The animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("The dog barks");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("The cat meows");
    }
}

public class TestPolymorphism {
    public static void main(String[] args) {
        Animal myAnimal;

        myAnimal = new Dog();
        myAnimal.makeSound(); // Output: The dog barks

        myAnimal = new Cat();
        myAnimal.makeSound(); // Output: The cat meows
    }
}

```

Super

La parola chiave `super` in Java ha diverse funzionalità e viene utilizzata principalmente in tre contesti:

1. **Chiamare il costruttore della superclasse.**
2. **Accedere a metodi della superclasse che sono stati sovrascritti nella sottoclasse.**
3. **Accedere ai membri della superclasse (attributi) che sono stati nascosti dalla sottoclasse.**

Chiamare il costruttore della superclasse

Quando si crea un'istanza di una sottoclasse, il costruttore della sottoclasse può chiamare il costruttore della superclasse usando `super()`. Questo è spesso necessario per inizializzare la parte della superclasse dell'oggetto.

Esempio:

```

class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // Calls the constructor of Animal
        System.out.println("Dog constructor");
    }
}

public class TestSuperConstructor {
    public static void main(String[] args) {
        Dog dog = new Dog();
        // Output:
        // Animal constructor
        // Dog constructor
    }
}

```

Accedere a metodi della superclasse sovrascritti nella sottoclasse

Se un metodo nella sottoclasse sovrascrive un metodo nella superclasse, si può usare `super` per chiamare il metodo della superclasse.

Esempio:

```

class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        super.makeSound(); // Calls the makeSound method of Animal
        System.out.println("Dog barks");
    }
}

public class TestSuperMethod {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        // Output:
    }
}

```

```
        // Animal sound
        // Dog barks
    }
}
```

Accedere ai membri della superclasse nascosti nella sottoclasse

Se un attributo della sottoclasse nasconde un attributo della superclasse (cioè hanno lo stesso nome), si può usare `super` per accedere all'attributo della superclasse.

Esempio:

```
class Animal {
    String color = "white";
}

class Dog extends Animal {
    String color = "black";

    void printColor() {
        System.out.println("Dog color: " + color); // prints color of Dog class
        System.out.println("Animal color: " + super.color); // prints color of Animal
    }
}

class
}

public class TestSuperAttribute {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.printColor();
        // Output:
        // Dog color: black
        // Animal color: white
    }
}
```

In sintesi, `super` è una parola chiave utile in Java per interagire con la superclasse da una sottoclasse, permettendo la chiamata a costruttori, metodi e l'accesso a membri che potrebbero essere nascosti dalla sottoclasse.

La classe Object

In Java, la classe `Object` è la classe radice di tutte le classi. Ogni classe definita implicitamente o esplicitamente eredita da `Object`. Ciò significa che ogni oggetto in Java ha come antenato la classe `Object`. Questo fornisce a tutti gli oggetti una serie di metodi comuni che possono essere utilizzati in modo universale

Esempio di Override dei Metodi di `Object`

Ecco un esempio su come si possono sovrascrivere `equals`, `hashCode` e `toString` in una classe personalizzata:

```
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }
        Person person = (Person) obj;
        return age == person.age && Objects.equals(name, person.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + '}';
    }
}
```

In questo esempio, abbiamo creato una classe `Person` con due attributi: `name` e `age`. Abbiamo sovrascritto `equals` per confrontare gli attributi degli oggetti `Person`, `hashCode` per generare un valore hash basato su questi attributi e `toString` per fornire una rappresentazione leggibile della classe `Person`.

Metodi Principali della Classe `Object`

1. `public boolean equals(Object obj)`
 - Confronta questo oggetto con l'oggetto specificato per l'uguaglianza. Di default, `equals` confronta i riferimenti di memoria, quindi due oggetti sono uguali solo se sono lo stesso oggetto. Può essere sovrascritto per fornire una logica di confronto personalizzata.
2. `protected Object clone() throws CloneNotSupportedException`
 - Crea e ritorna una copia dell'oggetto. La classe deve implementare l'interfaccia `Cloneable` e sovrascrivere `clone` per supportare la clonazione.
3. `public String toString()`
 - Ritorna una rappresentazione stringa dell'oggetto. Di default, ritorna una stringa che consiste del nome della classe seguito dal simbolo `@` e l'hashcode dell'oggetto. Questo metodo è comunemente sovrascritto per fornire una rappresentazione più leggibile e utile.

Struttura del metodo `equals`

Il metodo `equals` in Java è utilizzato per confrontare due oggetti e verificare se sono "uguali" secondo una definizione specifica. È definito nella classe base `Object` e può essere sovrascritto per implementare logiche di confronto personalizzate. Di default, `equals` usa il confronto di riferimento (ovvero, verifica se due variabili fanno riferimento allo stesso oggetto in memoria), ma in molti casi è utile sovrascriverlo per verificare l'equivalenza logica dei valori degli oggetti.

Il metodo `equals` è definito come segue nella classe `Object`:

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

Questa implementazione predefinita restituisce `true` solo se `this` e `obj` puntano allo stesso oggetto. Tuttavia, per confrontare due oggetti in base ai loro campi, è necessario sovrascrivere `equals`.

Sovrascrittura del metodo `equals`

Quando sovrascrivi `equals`, assicurati di seguire queste linee guida per implementare un confronto efficace:

1. **Confronta la stessa istanza:** verifica se `this` e `obj` fanno riferimento allo stesso oggetto in memoria.
2. **Verifica che `obj` non sia `null`:** se `obj` è `null`, restituisci `false`.
3. **Confronta la classe:** assicurati che `obj` sia un'istanza della stessa classe della tua classe corrente.
4. **Confronta i campi rilevanti:** verifica se i campi significativi per il confronto sono uguali.

Ecco un esempio di implementazione di `equals` per una classe `Person`:

```

public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object obj) {
        // 1. Controllo dell'istanza stessa
        if (this == obj) return true;

        // 2. Verifica che l'oggetto non sia null e che appartenga alla stessa classe
        if (obj == null || getClass() != obj.getClass()) return false;

        // 3. Confronto dei campi significativi
        Person person = (Person) obj;
        return age == person.age && (name != null ? name.equals(person.name) :
person.name == null);
    }
}

```

Regole importanti per `equals`

Quando sovrascrivi `equals`, assicurati che la tua implementazione rispetti queste proprietà:

1. **Riflessività:** un oggetto deve essere uguale a sé stesso. Ovvero, `x.equals(x)` deve restituire `true`.
2. **Simmetria:** se `x.equals(y)` è `true`, allora anche `y.equals(x)` deve essere `true`.
3. **Transitività:** se `x.equals(y)` e `y.equals(z)` sono entrambi `true`, allora `x.equals(z)` deve essere `true`.
4. **Consistenza:** chiamate multiple a `equals` devono restituire lo stesso valore finché gli oggetti non cambiano.
5. **Nullità:** un confronto con `null` deve restituire `false` (ovvero, `x.equals(null)` restituisce sempre `false`).

Uso del metodo `equals` e del metodo `hashCode`

Quando si sovrascrive `equals`, è buona pratica sovrascrivere anche `hashCode`. Questo è particolarmente importante se l'oggetto verrà utilizzato in strutture di dati come `HashMap` o `HashSet`, che usano `hashCode` per ottimizzare le ricerche e la gestione dei dati.

Il metodo `hashCode` in Java serve a generare un valore numerico (il *codice hash*) che rappresenta un'istanza di un oggetto. Questo codice hash è un intero derivato dagli attributi dell'oggetto ed è usato nelle strutture di dati basate su hash, come `HashMap`, `HashSet`, e `HashTable`, per ottimizzare le operazioni di ricerca, inserimento e rimozione.

Come funziona il `hashCode`

Il `hashCode` consente alle strutture dati di posizionare l'oggetto in una determinata "bucket" o "cella" che ne rappresenta una posizione ideale in base al suo valore hash. Questo valore viene utilizzato per accedere direttamente all'oggetto, anziché dover scorrere tutta la collezione.

Relazione tra `hashCode` ed `equals`

`hashCode` ed `equals` sono strettamente legati e, per garantire il corretto funzionamento delle strutture dati basate su hash, devono rispettare la seguente regola:

- Se due oggetti sono uguali secondo il metodo `equals`, devono avere lo stesso `hashCode`.
- Se due oggetti hanno lo stesso `hashCode`, non è necessario che siano uguali (questo fenomeno è noto come *collisione hash*).

Se non rispetti questa regola sovrascrivendo `hashCode` e `equals` in modo inconsistente, potresti ottenere comportamenti imprevisti nelle strutture dati basate su hash, come l'incapacità di trovare elementi correttamente.

Sovrascrivere il metodo `hashCode`

Per sovrascrivere `hashCode`, si calcola un codice hash basato sui campi significativi dell'oggetto. Java fornisce il metodo `Objects.hash(...)` per generare facilmente un `hashCode` affidabile. Ecco un esempio:

```
import java.util.Objects;

public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Person person = (Person) obj;
        return age == person.age && name.equals(person.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age); // Crea un hash basato su name e age
    }
}
```

Importanza del metodo `hashCode`

Ecco perché il metodo `hashCode` è fondamentale:

1. **Efficienza:** rende più efficienti le operazioni di ricerca, inserimento e cancellazione nelle strutture dati come `HashMap` e `HashSet`.
2. **Correttezza:** quando `equals` è sovrascritto, è necessario sovrascrivere `hashCode` per mantenere la coerenza e rispettare il contratto tra `equals` e `hashCode`.
3. **Riduzione delle collisioni:** una buona implementazione di `hashCode` distribuisce gli oggetti in modo uniforme, riducendo le collisioni e migliorando le prestazioni delle strutture dati.

Senza `hashCode`, o con una sua implementazione inconsistente, l'uso di oggetti in `HashSet` o `HashMap` può diventare inefficiente e portare a risultati inattesi, come duplicati indesiderati o difficoltà nel reperire gli oggetti stessi.

Struttura del metodo `getClass`

Il metodo `getClass()` è un metodo della classe base `Object` in Java, che viene ereditato da tutte le classi. Questo metodo restituisce un oggetto di tipo `Class<?>`, che rappresenta la classe runtime dell'oggetto su cui viene chiamato.

Firma del Metodo

```
public final Class<?> getClass()
```

- **public**: È accessibile da qualsiasi classe.
- **final**: Non può essere sovrascritto da sottoclassi.
- **Class<?>**: Restituisce un oggetto che descrive la classe dell'oggetto runtime.

Funzionamento

1. **Restituisce il Tipo di Classe Runtime:** Anche se un oggetto è dichiarato come una classe base, `getClass()` restituisce la classe runtime effettiva (quella reale dell'istanza).
2. **Informazioni sulla Classe:** Puoi usare l'oggetto `Class` per accedere a informazioni come il nome della classe, il pacchetto, i metodi e i campi.

Esempio di Utilizzo

Esempio Base:

```
public class Main {
    public static void main(String[] args) {
        Object obj = new String("Hello, World!");

        // Usa getClass per ottenere la classe runtime
        Class<?> clazz = obj.getClass();

        System.out.println("Class name: " + clazz.getName());
        System.out.println("Simple class name: " + clazz.getSimpleName());
        System.out.println("Package: " + clazz.getPackage());
    }
}
```

Output:

```
Class name: java.lang.String
Simple class name: String
Package: package java.lang
```

Descrizione:

1. `getClass().getName()`: Restituisce il nome completo della classe (incluso il package).
2. `getClass().getSimpleName()`: Restituisce solo il nome della classe senza il package.
3. `getClass().getPackage()`: Restituisce il package della classe.

Esempio con Ereditarietà

```
class Animal {}
class Dog extends Animal {}

public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Dog();

        // Ottieni la classe runtime dell'oggetto
        System.out.println("Class of myAnimal: " + myAnimal.getClass().getName());
    }
}
```

Output:

```
Class of myAnimal: Dog
```

Descrizione:

- Anche se l'oggetto `myAnimal` è dichiarato come `Animal`, il metodo `getClass()` restituisce `Dog`, che è la classe runtime effettiva.

Applicazioni Pratiche

1. Reflection

:Il metodo `getClass()` è ampiamente utilizzato in **Java Reflection API** per ottenere informazioni sulle classi e lavorare dinamicamente con metodi e campi.

2. Debug e Logging

:Puoi usare `getClass()` per registrare il tipo di classe degli oggetti durante il debug.

3. Controlli di Classe

:Confronti tra tipi di oggetti o verifica se un oggetto appartiene a una determinata classe.

Nota Importante

Il metodo `getClass()` restituisce sempre la classe runtime effettiva e non considera eventuali sottoclassi anonime o proxy. Se hai bisogno di controllare l'ereditarietà, puoi utilizzare:

- `instanceof` per verificare l'appartenenza.
- Metodi come `Class.isAssignableFrom()`.

Conclusione

Il metodo `getClass()` è una funzionalità fondamentale di Java che consente di accedere alle informazioni runtime degli oggetti, rendendolo utile per debug, log, e scenari avanzati come il reflection.

Classe Astratta in Java

In Java, una **classe astratta** è una classe che non può essere istanziata direttamente. Viene utilizzata come modello per altre classi e può contenere metodi astratti (senza implementazione) e metodi concreti (con implementazione). Le classi che estendono la classe astratta devono implementare tutti i metodi astratti definiti nella classe astratta, a meno che non siano anch'esse astratte.

Esempio di Classe Astratta:

```
abstract class Animal {
    // Abstract method (no implementation)
    public abstract void sound();

    // Concrete method (with implementation)
    public void sleep() {
        System.out.println("The animal is sleeping");
    }
}

class Dog extends Animal {
    // Implementation of the abstract method
    public void sound() {
        System.out.println("The dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        // Animal animal = new Animal(); // Error: Cannot instantiate abstract class
        Animal dog = new Dog();
        dog.sound(); // Output: The dog barks
        dog.sleep(); // Output: The animal is sleeping
    }
}
```

Interfaccia in Java

Un'**interfaccia** in Java è un contratto che specifica un insieme di metodi che una classe deve implementare. A differenza delle classi astratte, un'interfaccia non può contenere implementazioni di metodi (prima di Java 8). A partire da Java 8, è possibile definire metodi predefiniti con implementazioni all'interno di un'interfaccia usando il modificatore `default`.

Caratteristiche:

- Tutti i metodi in un'interfaccia sono implicitamente `abstract` (prima di Java 8).
- Le classi che implementano un'interfaccia devono fornire un'implementazione per tutti i metodi dichiarati nell'interfaccia.
- Una classe può implementare più interfacce, ma può estendere solo una classe.

Esempio di Interfaccia:

```
interface Animal {
    // Abstract method
    void sound();
}
```

```

}

class Cat implements Animal {
    // Implementation of the abstract method
    public void sound() {
        System.out.println("The cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal cat = new Cat();
        cat.sound(); // Output: The cat meows
    }
}

```

In sintesi, una **classe astratta** può contenere sia metodi astratti sia concreti, mentre un'**interfaccia** (fino a Java 8) contiene solo metodi astratti. Le classi possono implementare più interfacce ma possono estendere una sola classe.

I Generici in Java

I generici permettono di scrivere codice che funziona con tipi diversi mantenendo la sicurezza dei tipi a tempo di compilazione. Sono uno degli strumenti più importanti del linguaggio e li usi già ogni volta che lavori con le collezioni.

Il problema che risolvono

Senza generici, una classe "scatola" che contiene un oggetto qualsiasi dovrebbe usare `Object`:

```

public class Box {
    private Object content;

    public void set(Object content) { this.content = content; }
    public Object get() { return content; }
}

```

Funziona, ma ha due problemi seri:

```

Box box = new Box();
box.set("Hello");

String text = (String) box.get(); // cast manuale obbligatorio
box.set(42);                       // accetta qualsiasi tipo
String oops = (String) box.get(); // ClassCastException a runtime

```

Il compilatore non aiuta: l'errore emerge solo quando il programma è in esecuzione.

La soluzione: parametri di tipo

I generici permettono di dichiarare un *segnaposto* per il tipo, indicato tra parentesi angolari:

```
public class Box<T> {  
    private T content;  
  
    public void set(T content) { this.content = content; }  
    public T get() { return content; }  
}
```

Quando si crea l'oggetto, si decide quale tipo concreto sostituire a `T`:

```
Box<String> stringBox = new Box<>();  
stringBox.set("Hello");  
String text = stringBox.get(); // nessun cast  
  
stringBox.set(42); // errore di compilazione  
  
Box<Integer> intBox = new Box<>();  
intBox.set(42);
```

Gli errori vengono segnalati durante la scrittura del codice, non a runtime.

Convenzioni sui nomi

Le lettere usate per i parametri di tipo seguono convenzioni comuni:

T — Type

Un tipo generico qualsiasi.

E — Element

L'elemento di una collezione (`List<E>`, `Set<E>`).

K e V — Key e Value

La chiave e il valore in una mappa (`Map<K, V>`).

N — Number

Un tipo numerico.

Generici nelle collezioni

I generici si incontrano quotidianamente nelle collezioni della libreria standard:

```
List<String> names = new ArrayList<>();
names.add("Mario");
names.add("Luigi");
String first = names.get(0); // già una String, niente cast

Map<String, Integer> ages = new HashMap<>();
ages.put("Mario", 30);
int marioAge = ages.get("Mario");
```

`List<String>` significa "lista che contiene solo `String`". Aggiungere un numero produce un errore di compilazione.

Il diamond operator

La sintassi `<>` con parentesi vuote permette di omettere il tipo a destra, perché il compilatore lo deduce dalla dichiarazione:

```
List<String> names = new ArrayList<String>(); // forma esplicita
List<String> names = new ArrayList<>();      // forma compatta
```

Metodi generici

Un singolo metodo può essere generico anche se la classe non lo è. Il parametro di tipo si dichiara prima del tipo di ritorno:

```
public class Utils {

    public static <T> T firstElement(List<T> list) {
        return list.get(0);
    }
}
```

Il compilatore deduce automaticamente il tipo dall'argomento:

```
List<String> names = List.of("Mario", "Luigi");
String first = Utils.firstElement(names); // T diventa String

List<Integer> numbers = List.of(1, 2, 3);
Integer firstNum = Utils.firstElement(numbers); // T diventa Integer
```

Vincolare il tipo con extends

Quando il metodo deve poter chiamare metodi specifici sul parametro, si vincola `T` a un tipo o a una sua sottoclasse:

```
public static <T extends Number> double sum(List<T> values) {  
    double total = 0;  
    for (T value : values) {  
        total += value.doubleValue();  
    }  
    return total;  
}
```

`<T extends Number>` significa "T deve essere `Number` o un suo sottotipo" (`Integer`, `Double`, `Long`, ecc.). Senza il vincolo, il compilatore tratterebbe `T` come `Object` e `doubleValue()` non sarebbe accessibile.

Più parametri di tipo

Una classe può dichiarare più parametri di tipo separati da virgole. Esempio: una classe `Pair` che tiene insieme due valori di tipi potenzialmente diversi:

```
public class Pair<A, B> {  
    private final A first;  
    private final B second;  
  
    public Pair(A first, B second) {  
        this.first = first;  
        this.second = second;  
    }  
  
    public A getFirst() { return first; }  
    public B getSecond() { return second; }  
}
```

Uso:

```
Pair<String, Integer> person = new Pair<>("Mario", 30);  
String name = person.getFirst();  
int age = person.getSecond();  
  
Pair<String, String> coords = new Pair<>("Lat", "Long");
```

La stessa classe funziona con qualsiasi combinazione di tipi.

Vantaggi principali

Riutilizzo del codice

Una sola classe o metodo funziona con tipi diversi senza duplicazioni.

Type safety a compile-time

Gli errori di tipo emergono durante la compilazione, non a runtime.

Eliminazione dei cast manuali

Il codice è più pulito e leggibile, senza conversioni esplicite.

Cosa ricordare

I generici sono ovunque nel codice Java moderno: ogni `List`, `Map`, `Optional` o `Stream` li utilizza. Comprenderne le basi è il primo passo per leggere e scrivere codice idiomático. Argomenti più avanzati come le *wildcards* (`?`, `? extends`, `? super`) e la *type erasure* si affrontano dopo aver acquisito familiarità con queste fondamenta.

Thread

Un **thread** è un flusso di esecuzione indipendente all'interno di un programma.

Quando un programma Java parte, esiste già un thread principale chiamato **main thread**.

Perché creare un nuovo thread

Creo un nuovo thread quando voglio eseguire un'attività senza bloccare il resto del programma.

Il caso tipico è questo: il programma deve fare un'operazione lunga, lenta o imprevedibile, ma nel frattempo deve continuare a rispondere, elaborare altre istruzioni o gestire altri utenti.

Un thread permette quindi di separare il **flusso principale** del programma da uno o più **flussi secondari**.

Thread principale e thread secondari

Quando un programma Java parte, esiste già un thread principale: il **main thread**.

Il main thread esegue il metodo `main` e rappresenta il flusso iniziale del programma.

Quando creiamo un nuovo thread, stiamo dicendo al programma:

questa attività può essere eseguita separatamente rispetto al flusso principale.

In questo modo il main thread può continuare il proprio lavoro, mentre un altro thread svolge un'attività diversa.

A cosa serve concretamente

Un thread serve soprattutto a gestire attività che richiedono tempo o che potrebbero rimanere in attesa.

Per esempio:

- lettura o scrittura di file grandi
- chiamate verso API esterne
- download o upload di dati
- invio di email
- elaborazioni pesanti
- gestione di più utenti o richieste contemporanee
- operazioni in background
- aggiornamenti periodici

Senza thread, una di queste attività potrebbe bloccare tutto il programma.

Il concetto di blocco

Un programma si “blocca” quando resta fermo in attesa che un’operazione finisca.

Non significa necessariamente che il programma sia andato in errore. Significa che, fino a quando quell’operazione non termina, il flusso corrente non può andare avanti.

Esempio concettuale:

1. il programma inizia
2. deve scaricare un file
3. aspetta che il download finisca
4. solo dopo continua

Se il download dura 10 secondi, il programma resta fermo 10 secondi.

Con un thread separato, invece, il download può avvenire in background mentre il programma continua a fare altro.

Thread e programmi più reattivi

Uno dei motivi principali per usare i thread è rendere il programma più reattivo.

Questo è particolarmente importante nelle applicazioni con interfaccia grafica.

Se un’applicazione desktop esegue un’operazione pesante nello stesso thread che gestisce l’interfaccia, la finestra può sembrare congelata: i pulsanti non rispondono, il caricamento non si aggiorna, l’utente ha l’impressione che il programma sia bloccato.

Separando il lavoro pesante in un thread secondario, l’interfaccia può continuare a rispondere.

Thread e server

I thread sono molto importanti anche nei server.

Un server deve spesso gestire più richieste nello stesso momento.

Se un server potesse gestire una sola richiesta alla volta, ogni utente dovrebbe aspettare la fine della richiesta precedente.

Con i thread, invece, il server può gestire più operazioni contemporaneamente o in modo concorrente.

Questo permette a più utenti di usare lo stesso servizio nello stesso momento.

Thread e attività in background

Un altro caso comune è l'esecuzione di attività in background.

Sono attività che non devono necessariamente bloccare l'utente o il flusso principale del programma.

Esempi:

- sincronizzare dati
- controllare notifiche
- generare report
- salvare log
- pulire file temporanei
- aggiornare una cache
- eseguire controlli periodici

Queste attività possono essere svolte mentre il programma continua a funzionare normalmente.

Thread e performance

È importante chiarire una cosa agli studenti: usare i thread non significa automaticamente rendere un programma più veloce.

I thread aiutano soprattutto a:

- non bloccare il programma
- eseguire attività in modo concorrente
- sfruttare meglio il processore quando ci sono più core disponibili
- organizzare meglio lavori indipendenti

Però creare troppi thread può peggiorare le prestazioni.

Ogni thread ha un costo: occupa memoria, deve essere gestito dal sistema operativo e richiede tempo per passare da un thread all'altro.

Quindi i thread vanno usati quando servono davvero.

Concorrenza e parallelismo

Quando si parla di thread bisogna distinguere due concetti: **concorrenza** e **parallelismo**.

La **concorrenza** significa che più attività avanzano nello stesso periodo di tempo, anche se non necessariamente nello stesso identico istante.

Il **parallelismo** significa che più attività vengono eseguite realmente nello stesso momento, per esempio su più core del processore.

Un programma con più thread può essere concorrente. Può diventare anche parallelo se il sistema ha risorse hardware sufficienti.

Rischi nell'uso dei thread

I thread sono utili, ma introducono anche complessità.

Il problema principale nasce quando più thread accedono agli stessi dati.

Se due thread modificano la stessa variabile o la stessa struttura dati nello stesso momento, possono nascere comportamenti imprevedibili.

Questi problemi si chiamano spesso:

- race condition
- problemi di sincronizzazione
- dati incoerenti
- deadlock

Per questo, nei programmi reali, i thread vanno gestiti con attenzione.

Casi comuni in Java

In Java i thread vengono usati spesso in questi scenari:

Applicazioni desktop

Per non bloccare l'interfaccia grafica durante caricamenti, elaborazioni o operazioni lente.

Applicazioni server

Per gestire più richieste o più client contemporaneamente.

Operazioni di input/output

Per leggere file, scrivere file, chiamare API, scaricare dati o comunicare con database senza bloccare il flusso principale.

Elaborazioni pesanti

Per dividere lavori lunghi o complessi, come generazione di report, analisi dati o trasformazione di immagini.

Task periodici

Per eseguire controlli, sincronizzazioni, pulizie o aggiornamenti a intervalli regolari.

Background jobs

Per svolgere attività secondarie mentre il programma principale continua a funzionare.

Quando non conviene creare un thread

Non conviene creare un nuovo thread per operazioni semplici, immediate o molto brevi.

Se un'attività dura pochissimo, il costo di creare e gestire il thread può essere superiore al beneficio.

Inoltre non conviene creare thread manualmente in modo incontrollato, soprattutto nei programmi grandi.

In molti casi reali si usano strumenti più evoluti, come pool di thread, executor o framework che gestiscono i thread automaticamente.

Messaggio chiave

Un thread serve per permettere a un programma di fare più attività nello stesso periodo di tempo.

Il motivo principale per creare un nuovo thread non è “andare più veloce”, ma:

evitare che un'attività lunga o bloccante fermi tutto il programma.

Quindi possiamo dire:

- il main thread gestisce il flusso principale
- i thread secondari gestiscono lavori separati
- `run()` contiene il lavoro da eseguire
- `start()` avvia davvero un nuovo thread

Questa è la base per capire la programmazione concorrente in Java.

Creare un thread con `extends Thread`

In questo caso la classe eredita dalla classe `Thread`.

Il lavoro che il thread deve eseguire viene scritto nel metodo `run()`.

```
class MyThread extends Thread {
```

```
@Override
public void run() {
    System.out.println("Thread is running");
}

}

public class Main {

    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start();
    }
}
```

Come funziona

Quando viene chiamato:

```
thread.start();
```

Java:

1. crea un nuovo thread
2. esegue automaticamente il metodo `run()`

Creare un thread con `Runnable`

Con `Runnable` separiamo:

- il lavoro da eseguire
- il thread che lo esegue

```
class MyRunnable implements Runnable {

    @Override
    public void run() {
        System.out.println("Runnable is running");
    }
}

public class Main {

    public static void main(String[] args) {

        MyRunnable task = new MyRunnable();
        Thread thread = new Thread(task);
```

```
        thread.start();
    }
}
```

Differenza tra `start()` e `run()`

Questa è la parte fondamentale.

`start()`

```
thread.start();
```

- crea un nuovo thread
- esegue `run()` in parallelo al main thread

`run()`

```
thread.run();
```

NON crea un nuovo thread.

È semplicemente una normale chiamata di metodo.

Esempio pratico

```
class MyThread extends Thread {

    @Override
    public void run() {
        System.out.println(
            "Executed by: " + Thread.currentThread().getName()
        );
    }
}

public class Main {

    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.run();
        thread.start();
    }
}
```

Possibile output

```
Executed by: main
Executed by: Thread-0
```

Interpretazione

Quando scriviamo:

```
thread.run();
```

il metodo viene eseguito dal thread principale (`main`).

Quando scriviamo:

```
thread.start();
```

Java crea un nuovo thread (`Thread-0`).

Differenza concettuale

Metodo	Comportamento
<code>run()</code>	normale chiamata di metodo
<code>start()</code>	crea un nuovo thread

Frase semplice da ricordare

`run()` contiene il lavoro da fare.

`start()` crea davvero il nuovo thread.

Eccezioni in Java

In Java, un'**eccezione** è un evento anomalo che si verifica durante l'esecuzione di un programma e che interrompe il normale flusso delle istruzioni.

Esempi tipici:

```
int result = 10 / 0;
```

Questo codice genera un errore a runtime, perché non è possibile dividere per zero.

Oppure:

```
String name = null;
System.out.println(name.length());
```

Questo genera una `NullPointerException`, perché stiamo provando a usare un oggetto che in realtà non esiste.

Perché esistono le eccezioni?

Le eccezioni servono per evitare che il programma si interrompa in modo incontrollato.

Senza un meccanismo di gestione degli errori, un problema improvviso potrebbe bloccare completamente l'applicazione.

Con le eccezioni, invece, possiamo dire:

“Prova a eseguire questo codice. Se qualcosa va storto, gestisci il problema in questo modo.”

Il blocco `try-catch`

La gestione più semplice delle eccezioni avviene con `try` e `catch`.

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException exception) {
            System.out.println("You cannot divide by zero.");
        }

        System.out.println("Program continues...");
    }
}
```

Il blocco `try` contiene il codice che potrebbe generare un'eccezione.

Il blocco `catch` contiene il codice da eseguire se quella specifica eccezione si verifica.

In questo esempio, il programma non si blocca. Stampa un messaggio di errore e poi continua.

Cosa succede quando viene lanciata un'eccezione?

Quando Java incontra un problema, crea un oggetto eccezione.

Nel caso della divisione per zero, viene creato un oggetto di tipo:

```
ArithmeticException
```

A quel punto Java cerca un blocco `catch` compatibile con quel tipo di eccezione.

Se lo trova, esegue il codice dentro il `catch`.

Se non lo trova, il programma termina con un errore.

Il flusso di esecuzione

Guarda questo esempio:

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Before error");  
  
            int result = 10 / 0;  
  
            System.out.println("After error");  
        } catch (ArithmeticException exception) {  
            System.out.println("Error handled");  
        }  
  
        System.out.println("End of program");  
    }  
}
```

Output:

```
Before error  
Error handled  
End of program
```

La riga:

```
System.out.println("After error");
```

non viene mai eseguita.

Questo perché, appena si verifica l'eccezione, Java interrompe il blocco `try` e passa direttamente al `catch`.

Gestire più tipi di eccezioni

Un blocco `try` può avere più blocchi `catch`.

```
public class Main {
    public static void main(String[] args) {
        try {
            String text = null;
            System.out.println(text.length());
        } catch (ArithmeticException exception) {
            System.out.println("Math error");
        } catch (NullPointerException exception) {
            System.out.println("Object is null");
        } catch (Exception exception) {
            System.out.println("Generic error");
        }
    }
}
```

Java controlla i `catch` dall'alto verso il basso.

Per questo motivo, bisogna mettere prima le eccezioni più specifiche e poi quelle più generiche.

Quindi è corretto:

```
catch (NullPointerException exception)
catch (Exception exception)
```

Non è corretto fare il contrario, perché `Exception` è troppo generica e catturerebbe tutto prima degli altri blocchi.

Il blocco `finally`

Il blocco `finally` viene eseguito sempre, sia che l'eccezione si verifichi sia che non si verifichi.

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 2;
            System.out.println(result);
        } catch (ArithmeticException exception) {
            System.out.println("Math error");
        } finally {
            System.out.println("This code always runs");
        }
    }
}
```

`finally` si usa spesso per chiudere risorse, file, connessioni o stream.

Esempio concettuale:

```
try {  
    // Open file  
    // Read file  
} catch (Exception exception) {  
    // Handle error  
} finally {  
    // Close file  
}
```

Differenza tra `throw` e `throws`

Questi due termini sono simili, ma hanno significati diversi.

`throw`

`throw` serve per lanciare manualmente un'eccezione.

```
public class Main {  
    public static void main(String[] args) {  
        int age = 15;  
  
        if (age < 18) {  
            throw new IllegalArgumentException("User must be at least 18 years old.");  
        }  
  
        System.out.println("Access allowed");  
    }  
}
```

Qui siamo noi a decidere che, se l'età è minore di 18, il programma deve generare un'eccezione.

`throws`

`throws` si scrive nella dichiarazione di un metodo e indica che quel metodo potrebbe generare un'eccezione.

```
public static void readFile() throws IOException {  
    // Code that may generate an IOException  
}
```

Significa:

“Questo metodo potrebbe generare una `IOException`. Chi lo chiama deve gestirla.”

Checked e unchecked exceptions

In Java esistono due grandi categorie di eccezioni.

Checked exceptions

Sono eccezioni che Java obbliga a gestire.

Esempi:

```
IOException
FileNotFoundException
SQLException
```

Se un metodo può generare una checked exception, il compilatore obbliga il programmatore a fare una di queste due cose:

gestirla con `try-catch` oppure dichiararla con `throws`.

Esempio:

```
public static void readFile() throws IOException {
    // File reading logic
}
```

Oppure:

```
try {
    readFile();
} catch (IOException exception) {
    System.out.println("File error");
}
```

Unchecked exceptions

Sono eccezioni che Java non obbliga a gestire.

Esempi:

```
NullPointerException
ArithmeticException
ArrayIndexOutOfBoundsException
IllegalArgumentException
```

Di solito indicano errori logici nel codice.

Esempio:

```
int[] numbers = {1, 2, 3};
System.out.println(numbers[10]);
```

Questo genera una `ArrayIndexOutOfBoundsException`, perché l'indice 10 non esiste.

Gerarchia semplificata delle eccezioni

La struttura principale è questa:

```
Throwable
├── Error
└── Exception
    ├── RuntimeException
    │   ├── NullPointerException
    │   ├── ArithmeticException
    │   └── IllegalArgumentException
    └── IOException
```

`Throwable` è la classe padre di tutti gli errori ed eccezioni.

`Error` rappresenta problemi gravi della JVM o dell'ambiente di esecuzione. Di solito non si gestisce con `try-catch`.

`Exception` rappresenta problemi che un programma può prevedere e gestire.

`RuntimeException` rappresenta le eccezioni unchecked, cioè quelle non obbligatorie da gestire.

Esempio completo

```
public class Main {
    public static void main(String[] args) {
        try {
            int number = parseAge("abc");
            System.out.println("Age: " + number);
        } catch (NumberFormatException exception) {
            System.out.println("Invalid number format");
        } finally {
            System.out.println("Operation completed");
        }
    }

    public static int parseAge(String value) {
        return Integer.parseInt(value);
    }
}
```

Output:

```
Invalid number format
Operation completed
```

Il metodo `Integer.parseInt("abc")` prova a convertire una stringa in numero.

Ma `"abc"` non è un numero valido, quindi Java genera una `NumberFormatException`.

Il `catch` gestisce il problema e il `finally` viene comunque eseguito.

Eccezioni personalizzate

In Java possiamo anche creare eccezioni personalizzate.

```
public class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Poi possiamo usarla così:

```
public class Main {
    public static void main(String[] args) {
        try {
            registerUser(15);
        } catch (InvalidAgeException exception) {
            System.out.println(exception.getMessage());
        }
    }

    public static void registerUser(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("User must be at least 18 years old.");
        }

        System.out.println("User registered");
    }
}
```

Qui abbiamo creato una nostra eccezione chiamata `InvalidAgeException`.

È utile quando vogliamo rappresentare un errore specifico del nostro dominio applicativo.

Buone pratiche

Non bisogna usare le eccezioni per controllare il flusso normale del programma.

Meglio evitare codice di questo tipo:

```
try {  
    // Normal logic  
} catch (Exception exception) {  
    // Generic handling  
}
```

È preferibile catturare eccezioni specifiche:

```
catch (NumberFormatException exception)
```

invece di:

```
catch (Exception exception)
```

Inoltre, non bisogna ignorare silenziosamente un'eccezione:

```
catch (Exception exception) {  
}
```

Questo è pericoloso, perché nasconde il problema.

Meglio almeno stampare o loggare l'errore:

```
catch (Exception exception) {  
    System.out.println("Error: " + exception.getMessage());  
}
```

Riassunto

Un'eccezione è un problema che si verifica durante l'esecuzione del programma.

Java interrompe il flusso normale del codice e cerca un blocco `catch` capace di gestire quel problema.

Con `try` scriviamo il codice rischioso.

Con `catch` gestiamo l'errore.

Con `finally` eseguiamo codice che deve partire comunque.

Con `throw` lanciamo manualmente un'eccezione.

Con `throws` dichiariamo che un metodo può generare un'eccezione.

Le checked exceptions devono essere gestite obbligatoriamente.

Le unchecked exceptions non sono obbligatorie da gestire e spesso indicano errori di programmazione.

Il significato di `final` in Java

In Java, `final` è una keyword che indica che qualcosa **non può più essere modificato** o **non può essere ridefinito**, a seconda di dove viene usata.

Può essere applicata a:

1. variabili
2. parametri
3. attributi
4. metodi
5. classi

`final` su una variabile

Quando una variabile è dichiarata `final`, il suo valore può essere assegnato **una sola volta**.

```
public class Main {  
    public static void main(String[] args) {  
        final int age = 18;  
  
        // age = 20; // Error: cannot assign a value to final variable  
    }  
}
```

In questo caso `age` non può essere modificata dopo l'assegnazione iniziale.

Quindi `final` rende la variabile **non riassegnabile**.

`final` su una costante

Spesso `final` viene usato insieme a `static` per creare una costante.

```
public class MathConstants {  
    public static final double PI = 3.14159;  
}
```

Qui `PI` è:

- `static`, quindi appartiene alla classe e non ai singoli oggetti;
- `final`, quindi non può essere modificata.

Per convenzione, le costanti in Java si scrivono in maiuscolo.

```
public static final int MAX_USERS = 100;
```

`final` su un oggetto

Attenzione: se una variabile `final` contiene un oggetto, non significa che l'oggetto sia immutabile.

Significa solo che la variabile non può puntare a un altro oggetto.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        final ArrayList<String> names = new ArrayList<>();

        names.add("Mario");
        names.add("Luca");

        // names = new ArrayList<>(); // Error
    }
}
```

Qui posso modificare il contenuto della lista, perché l'oggetto è ancora modificabile.

Non posso però fare questo:

```
names = new ArrayList<>();
```

Perché `names` è `final` e non può essere riassegnata.

Quindi:

```
final
```

impedisce di cambiare il riferimento,
non necessariamente il contenuto dell'oggetto.

`final` su un parametro

Un parametro `final` non può essere riassegnato dentro il metodo.

```
public class Main {  
    public static void printName(final String name) {  
        System.out.println(name);  
  
        // name = "Other name"; // Error  
    }  
}
```

Questo può essere utile per rendere il codice più chiaro e per evitare modifiche accidentali al parametro.

final su un attributo

Un attributo `final` deve essere inizializzato una sola volta.

Può essere inizializzato direttamente:

```
public class User {  
    private final String name = "Mario";  
}
```

Oppure nel costruttore:

```
public class User {  
    private final String name;  
  
    public User(String name) {  
        this.name = name;  
    }  
}
```

Dopo l'inizializzazione, l'attributo non può più essere modificato.

```
public class User {  
    private final String name;  
  
    public User(String name) {  
        this.name = name;  
    }  
  
    public void changeName(String newName) {  
        // this.name = newName; // Error  
    }  
}
```

Questo è molto usato per creare classi più sicure e più prevedibili.

final su un metodo

Un metodo dichiarato `final` **non può essere sovrascritto** da una sottoclasse.

```
public class Animal {  
    public final void sleep() {  
        System.out.println("The animal is sleeping");  
    }  
}
```

Se una classe eredita da `Animal`, non può fare override del metodo `sleep`.

```
public class Dog extends Animal {  
    // Error: cannot override final method  
    /*  
    public void sleep() {  
        System.out.println("The dog is sleeping");  
    }  
    */  
}
```

Questo significa che il comportamento del metodo è bloccato.

La sottoclasse può usare il metodo, ma non può modificarne l'implementazione.

```
public class Dog extends Animal {  
    public void bark() {  
        System.out.println("Woof");  
    }  
}
```

Uso:

```
public class Main {  
    public static void main(String[] args) {  
        Dog dog = new Dog();  
  
        dog.sleep();  
        dog.bark();  
    }  
}
```

Output:

```
The animal is sleeping  
Woof
```

Quando ha senso usare un metodo `final`

Un metodo `final` si usa quando vogliamo impedire che una sottoclasse cambi un comportamento importante.

Esempio: immaginiamo una classe che gestisce un pagamento.

```
public class PaymentService {  
    public final void executePayment() {  
        validatePayment();  
        processPayment();  
        sendReceipt();  
    }  
  
    protected void validatePayment() {  
        System.out.println("Payment validated");  
    }  
  
    protected void processPayment() {  
        System.out.println("Payment processed");  
    }  
  
    protected void sendReceipt() {  
        System.out.println("Receipt sent");  
    }  
}
```

Qui `executePayment()` è `final`.

Questo vuol dire che nessuna sottoclasse può cambiare l'ordine delle operazioni:

1. validazione
2. pagamento
3. invio ricevuta

Le sottoclassi possono eventualmente personalizzare alcuni passaggi, ma non possono cambiare il flusso principale.

`final` su una classe

Una classe dichiarata `final` **non può essere estesa**.

```
public final class Utility {  
    public static void printMessage() {  
        System.out.println("Hello");  
    }  
}
```

Questa classe non può avere sottoclassi.

```
// Error: cannot inherit from final class
/*
public class MyUtility extends Utility {
}
*/
```

Un esempio famoso è `String`.

```
String text = "Hello";
```

La classe `String` è `final`, quindi nessuna classe può estenderla.

Questo serve anche per motivi di sicurezza, stabilità e immutabilità.

Differenza tra `final` su metodo e `final` su classe

```
public class Animal {
    public final void sleep() {
        System.out.println("Sleeping");
    }
}
```

Qui la classe `Animal` può essere estesa, ma il metodo `sleep()` non può essere sovrascritto.

Invece:

```
public final class Animal {
    public void sleep() {
        System.out.println("Sleeping");
    }
}
```

Qui è tutta la classe `Animal` a non poter essere estesa.

Quindi:

```
final su un metodo → impedisce l'override del metodo
final su una classe → impedisce l'ereditarietà della classe
```

Cosa `final` non permette di fare

Un metodo `final` non può essere anche `abstract`.

Questo non avrebbe senso:

```
public abstract class Animal {  
    // Error  
    /*  
    public abstract final void makeSound();  
    */  
}
```

Perché `abstract` significa:

Il metodo non ha implementazione e deve essere ridefinito da una sottoclasse.

Mentre `final` significa:

Il metodo non può essere ridefinito.

Sono quindi due concetti opposti.